

Human Activity Mode Detection Using an IMU

Measurement and Control Systems

Creator Name: Kian Amiri

Instructor: Prof. Nejat

July 2026

Contents

1	Introduction	3
2	Components	3
3	Circuit Schematic	3
4	Code Structure and Algorithm Explanation	5
4.1	Overview	5
4.2	Firmware (ESP32 / Arduino)	5
4.3	Python Scripts (Host Computer)	6
5	Experimental Setup and Test Procedure	6
6	Processed Results, Plots, and Calculations	7
6.1	Feature Summary	7
6.2	Classification Accuracy	7
7	Error Analysis and Limitations	8
8	Discussion of Repeatability and Reliability	8
9	Conclusion	9
A	Appendix: Full Source Code	9
A.1	ESP32 Firmware – Data Logger and Real-Time Classifier	9
A.2	Python – Serial Logger with Accuracy Evaluation	12

1 Introduction

Human activity recognition is a widely used measurement task in wearable electronics, fitness tracking, fall detection, and rehabilitation monitoring. Most practical systems rely on a small, low-cost inertial measurement unit (IMU) attached to the body, rather than a camera or an external tracking system, because the IMU is compact, inexpensive, and does not depend on line of sight.

The purpose of this project is to design and build a system that can distinguish between four everyday motion modes – walking, running, climbing stairs, and descending stairs – using only the acceleration and angular velocity signals from an IMU. An ESP32 microcontroller was chosen as the data-acquisition platform instead of a plain Arduino Uno because its built-in Wi-Fi hardware allows real-time wireless data transmission, which was one of the optional bonus items for this course.

The measurement task is important because it demonstrates the complete chain of a practical measurement system: sensing a physical quantity (acceleration and angular velocity), acquiring and labeling data, extracting meaningful features from noisy time-series signals, and using those features to make a real-time decision (classification) on embedded hardware with very limited computational resources. Unlike a machine-learning-based classifier, the system implemented here uses a simple, transparent, rule-based method, so that every decision the system makes can be traced back to a specific feature and threshold.

2 Components

Table 1 lists the exact hardware components used in this project.

Table 1: List of hardware components used in the project.

Component	Exact Model
Microcontroller	ESP32 development board (CP2102 USB-UART bridge, USB Type-C)
IMU sensor	MPU6050 (GY-521 breakout board)
Breadboard	MB-102 solderless breadboard
Wiring	Jumper wires (male-to-male / male-to-female)
Start/stop control	Touch-wire contact on pin D4 (replacing a physical push button)

3 Circuit Schematic

The MPU6050 communicates with the ESP32 using the I2C protocol. The exact connections used in the assembled circuit are summarized in Table 2.

Table 2: Pin connections between the ESP32 and the MPU6050 (GY-521).

ESP32 Pin	MPU6050 Pin	Function
3V3	VCC	Power supply (3.3 V)
GND (via negative rail)	GND	Common ground
D21	SDA	I2C data line
D22	SCL	I2C clock line
D4	– (touch-wire)	Digital input used as the recording start/stop control, configured with an internal pull-up resistor

The MPU6050 was confirmed to be correctly wired and functional using an I2C bus scanner, which detected the sensor at the expected address 0x68.

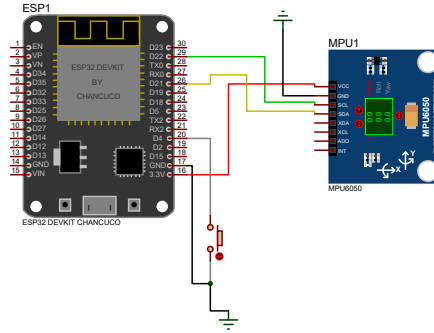


Figure 1: Circuit schematic of the ESP32–MPU6050 connection.

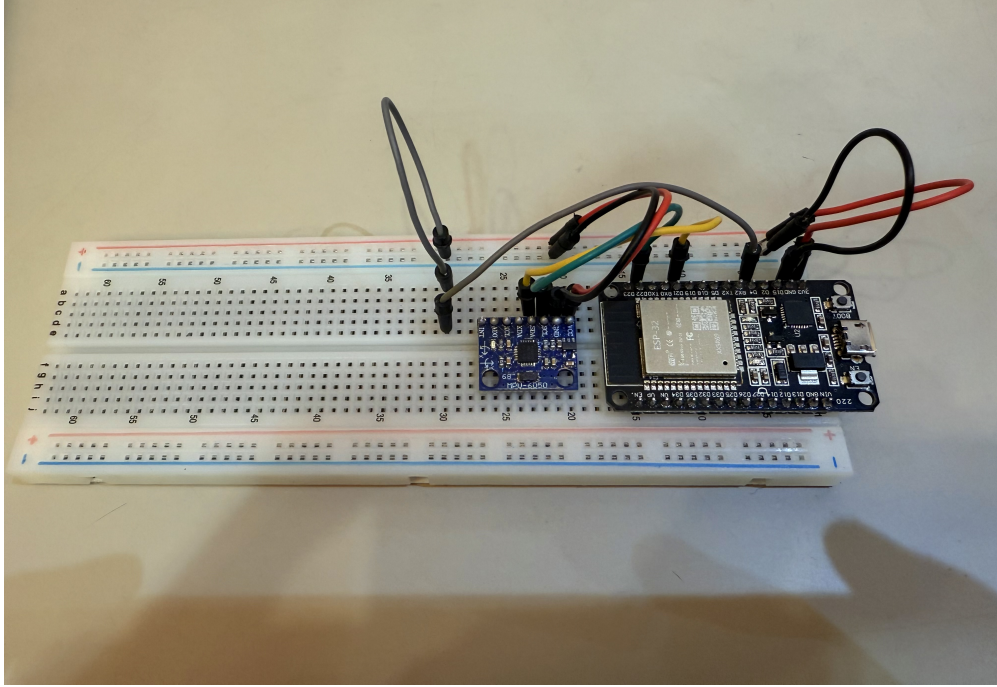


Figure 2: Photograph of the assembled hardware on the breadboard.

4 Code Structure and Algorithm Explanation

4.1 Overview

The software for this project was developed in two connected parts: (1) an ESP32 (Arduino) firmware responsible for sensor reading, labeled data logging, and real-time classification, and (2) a set of Python scripts running on a computer, responsible for serial data capture, offline feature extraction, and accuracy evaluation. The full source code is provided in the Appendix.

4.2 Firmware (ESP32 / Arduino)

The final firmware combines two previously separate development stages:

1. **Labeled data logging.** The activity label (e.g. Walking, Running, Up-stairs, Down-stairs) is set by typing it into the Serial Monitor. Recording is toggled on and off by briefly touching the wire connected to pin D4 (configured with INPUT_PULLUP), with software debouncing to avoid false triggers. While recording, the accelerometer and gyroscope values are streamed over serial as CSV rows in the format:

```
time_ms, label, accX, accY, accZ, gyroX, gyroY, gyroZ, detected
```

2. **Real-time feature extraction and classification.** The firmware keeps a sliding window of the last 40 samples (≈ 2 seconds at the 20 Hz sampling rate) for three signals: the Y-axis acceleration, the Z-axis acceleration, and the Y-axis angular velocity. For every new sample, once the window is full, three features are computed:

- Standard deviation of Y-axis acceleration (accY_std) – captures the overall intensity of vertical/forward motion.

- Standard deviation of Z-axis acceleration (`accZ_std`) – captures vertical bounce, which differs between level walking and stair climbing/descending.
 - Peak-to-peak value of Y-axis angular velocity (`gyroY_ptp`) – captures the range of forward/backward pitching motion, which is smaller during stair descent.
3. **Rule-based classification.** The three features are compared against fixed thresholds (derived from offline analysis of the recorded training data) using the following decision logic:
- If `accY_std` > 4.0: classified as **Running**.
 - Else if `accZ_std` < 0.7 and `gyroY_ptp` < 1.8: classified as **Down-stairs**.
 - Else if `accZ_std` < 1.1: classified as **Up-stairs**.
 - Otherwise: classified as **Walking**.

The predicted label (`detected`) is appended to each logged data row, alongside the ground-truth label typed by the user, so that classification accuracy can be computed offline.

4.3 Python Scripts (Host Computer)

- **Serial logger with accuracy evaluation.** Connects to the ESP32 over the serial port (115200 baud), forwards typed activity labels to the board, and saves every incoming data row to a CSV file. When the script is stopped, it automatically compares the `label` column (ground truth) against the `detected` column (real-time prediction) and computes the classification accuracy for each activity as well as the overall accuracy, saving the result to a summary CSV file.

5 Experimental Setup and Test Procedure

The MPU6050 sensor was mounted on the subject’s waist/chest using a strap, with its axes oriented consistently across all trials. Data was collected according to the following protocol:

- **Activities tested:** Walking, Running, Up-stairs, Down-stairs.
- **Repetitions:** 5 separate trials per activity.
- **Trial duration:** 20–30 seconds per trial.
- **Sampling rate:** approximately 20 Hz (50 ms sampling interval).
- **Procedure:** before each trial, the activity label was typed into the Serial Monitor / Python script. Recording was then started by touching the D4 control wire, the activity was performed continuously for the trial duration, and recording was stopped by touching the wire again. This was repeated 5 times for each of the 4 activities, giving 20 labeled trials in total.

No additional sensor calibration beyond the factory-default MPU6050 offsets was required, since the classification relies on statistical features of the signal (standard deviation, peak-to-peak) rather than on absolute acceleration or angle values.

6 Processed Results, Plots, and Calculations

6.1 Feature Summary

Table 3 summarizes the average value (across the 5 trials of each activity) of the three key features used by the classifier, together with the acceleration magnitude standard deviation for reference.

Table 3: Average feature values per activity, computed across 5 trials each.

Activity	accY_std	accZ_std	gyroY_ptp	accMag_std
Walking	1.546	2.475	1.771	1.563
Running	8.154	0.732	4.435	7.422
Down-stairs	1.845	2.256	1.629	1.864
Up-stairs	1.550	1.434	2.009	1.558

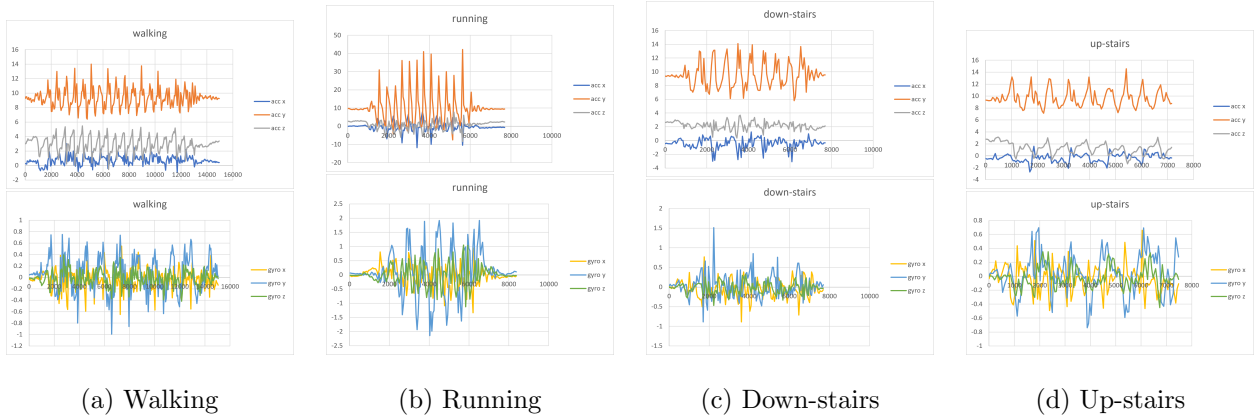


Figure 3: Comparing signals for each activity

6.2 Classification Accuracy

Table 4 presents the real-time classification accuracy, obtained by comparing the `detected` label produced by the firmware against the ground-truth `label` typed during data collection, for every 2-second sliding window.

Table 4: Real-time classification accuracy per activity.

Activity	Total windows	Correct	Accuracy (%)
Walking	545	495	90.83
Running	339	319	94.10
Up-stairs	502	344	68.53
Down-stairs	563	139	24.69
Overall	1949	1297	66.55

Running is detected almost perfectly, which is expected since its acceleration variability ($\text{accY_std} \approx 8.15$) is far above the other three activities and well separated by the fixed threshold of 4.0. Walking

is also detected reliably. Up-stairs classification is noticeably weaker, and Down-stairs classification is the weakest, which is analyzed further in Section 7.

7 Error Analysis and Limitations

Several sources of error affect the accuracy of the classification system:

- **Overlapping feature ranges.** Table 3 shows that the average `accZ_std` for Down-stairs (2.256) is actually *larger* than for Walking (2.475) is close to, and the `gyroY_ptp` ranges for Up-stairs (2.009) and Down-stairs (1.629) overlap with Walking (1.771). Because the rule-based thresholds (`DOWNSTAIRS_ACCZ_STD_MAX` = 0.7, `UPSTAIRS_ACCZ_STD_MAX` = 1.1) were tuned from trial-averaged data, individual windows within a trial can easily cross these boundaries in the wrong direction, which explains the low Down-stairs accuracy (24.69%).
- **Sensor placement sensitivity.** The IMU was strapped to the body rather than rigidly fixed, so small variations in orientation between trials directly change the measured acceleration components and shift the computed features.
- **Single-subject, single-orientation data.** All data was collected from one subject with one sensor mounting position and orientation. A rule-based threshold tuned this way is unlikely to generalize to a different person, walking style, or mounting location.
- **Fixed threshold vs. continuous variation.** Human motion is continuous and highly variable (e.g. stair descent can be performed at different speeds or with different degrees of caution), while the classifier relies on constant, hand-tuned thresholds that cannot adapt to this variability.
- **Sampling rate and window size.** A 20 Hz sampling rate and a 2-second window limit the frequency resolution and responsiveness of the detector; transitions between activities (e.g. stepping onto or off a staircase) are likely to be misclassified because the window still contains samples from the previous activity.
- **Sensor noise and quantization.** Like any low-cost MEMS IMU, the MPU6050 has intrinsic sensor noise, which directly affects the standard-deviation-based features used here.

8 Discussion of Repeatability and Reliability

Repeatability was assessed by comparing the feature values across the 5 independent trials recorded for each activity. Walking and Running trials showed relatively consistent `std/ptp` values from trial to trial (low trial-to-trial spread relative to the between-activity differences), which is consistent with their high classification accuracy (90.83% and 94.10%, respectively).

In contrast, the trial-to-trial variation for Up-stairs and Down-stairs was proportionally larger relative to the separation between their mean feature values and those of the neighboring classes. This indicates that the low accuracy observed for these two activities is not simply a threshold-tuning issue, but also reflects a genuine reliability limitation of the underlying measurement: single-axis, statistics-based features are not sufficiently distinctive to reliably separate Up-stairs and Down-stairs motion on a window-by-window basis, even though their trial-averaged values differ.

Overall, the system is repeatable and reliable for activities with strongly distinctive motion signatures (Walking, Running), but is not yet reliable enough for activities whose accelerometer/-gyroscope signatures are similar (Up-stairs vs. Down-stairs vs. Walking).

9 Conclusion

This project implemented a complete embedded measurement and classification pipeline using an ESP32 and an MPU6050 IMU. Labeled acceleration and angular-velocity data were collected for four activity modes, simple statistical features (standard deviation and peak-to-peak) were extracted using a sliding window, and a rule-based classifier running directly on the ESP32 achieved an overall real-time classification accuracy of 66.55%, with Running (94.10%) and Walking (90.83%) detected reliably, while Up-stairs (68.53%) and especially Down-stairs (24.69%) proved much harder to distinguish using the current feature set and fixed thresholds.

The main lesson learned is that even a small number of well-chosen features can separate motion modes with distinctive signatures very well, but simple fixed-threshold rules struggle when two activities produce statistically similar signals. The main limitations of the project are the reliance on a single subject and a single sensor mounting position, the use of only three scalar features, and the use of constant (non-adaptive) thresholds. Future improvements could include adding more features (e.g. dominant frequency via FFT, step rate), collecting data from multiple subjects and sensor placements, and/or replacing the fixed-threshold rules with a light-weight trained classifier (e.g. a decision tree) while keeping the same underlying feature set.

A Appendix: Full Source Code

A.1 ESP32 Firmware – Data Logger and Real-Time Classifier

```
1 #include <Adafruit_MPU6050.h>
2 #include <Adafruit_Sensor.h>
3 #include <Wire.h>
4 #include <math.h>
5
6 Adafruit_MPU6050 mpu;
7
8 const int buttonPin = 4; // D4 connected to button, other side to GND (using
   internal pull-up)
9
10 String activityLabel = "idle"; // default label before you set one
11 bool recording = false;
12 unsigned long startTime;
13
14 // Debounce for button
15 bool lastButtonState = HIGH;
16 unsigned long lastDebounceTime = 0;
17 const unsigned long debounceDelay = 50;
18
19 // ---- Sliding window for real-time detection ----
20 const int WINDOW_SIZE = 40; // ~2 seconds at 20 Hz
21 float accY_buf[WINDOW_SIZE];
22 float accZ_buf[WINDOW_SIZE];
23 float gyroY_buf[WINDOW_SIZE];
24 int bufIndex = 0;
25 bool bufFull = false;
26
27 // ---- Thresholds derived from collected data ----
28 const float RUNNING_ACCY_STD_THRESHOLD = 4.0;
29 const float DOWNSTAIRS_ACCZ_STD_MAX = 0.7;
30 const float UPSTAIRS_ACCZ_STD_MAX = 1.1;
31 const float DOWNSTAIRS_GYROY_PTP_MAX = 1.8;
```

```

32
33 float computeMean(float *buf, int n) {
34     float sum = 0;
35     for (int i = 0; i < n; i++) sum += buf[i];
36     return sum / n;
37 }
38
39 float computeStd(float *buf, int n, float mean) {
40     float sumSq = 0;
41     for (int i = 0; i < n; i++) {
42         float diff = buf[i] - mean;
43         sumSq += diff * diff;
44     }
45     return sqrt(sumSq / n);
46 }
47
48 float computePeakToPeak(float *buf, int n) {
49     float maxVal = buf[0];
50     float minVal = buf[0];
51     for (int i = 1; i < n; i++) {
52         if (buf[i] > maxVal) maxVal = buf[i];
53         if (buf[i] < minVal) minVal = buf[i];
54     }
55     return maxVal - minVal;
56 }
57
58 String classifyActivity(float accY_std, float accZ_std, float gyroY_ptp) {
59     if (accY_std > RUNNING_ACCY_STD_THRESHOLD) {
60         return "Running";
61     } else if (accZ_std < DOWNSTAIRS_ACCZ_STD_MAX && gyroY_ptp <
62         DOWNSTAIRS_GYROY_PTP_MAX) {
63         return "Down-stairs";
64     } else if (accZ_std < UPSTAIRS_ACCZ_STD_MAX) {
65         return "Up-stairs";
66     } else {
67         return "Walking";
68     }
69 }
70
71 void setup() {
72     Serial.begin(115200);
73     while (!Serial) delay(10);
74
75     pinMode(buttonPin, INPUT_PULLUP);
76
77     if (!mpu.begin()) {
78         Serial.println("Failed to find MPU6050 chip");
79         while (1) {
80             delay(10);
81         }
82     }
83
84     mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
85     mpu.setGyroRange(MPU6050_RANGE_500_DEG);
86     mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
87
88     delay(100);
89
90     Serial.println("Ready.");

```

```

90 Serial.println("Type a label (e.g. walking, running, upstairs, downstairs) and
    press Enter to set it.");
91 Serial.println("Press the button to start/stop recording.");
92 }
93
94 void loop() {
95     // --- Handle label input from Serial Monitor ---
96     if (Serial.available() > 0) {
97         String input = Serial.readStringUntil('\n');
98         input.trim();
99         if (input.length() > 0) {
100             activityLabel = input;
101             Serial.print("Label set to: ");
102             Serial.println(activityLabel);
103         }
104     }
105
106     // --- Handle button press (toggle recording) with debounce ---
107     bool reading = digitalRead(buttonPin);
108     if (reading != lastButtonState) {
109         lastDebounceTime = millis();
110     }
111
112     if ((millis() - lastDebounceTime) > debounceDelay) {
113         static bool buttonHandled = false;
114
115         if (reading == LOW && !buttonHandled) {
116             recording = !recording;
117             buttonHandled = true;
118
119             if (recording) {
120                 startTime = millis();
121                 bufIndex = 0;
122                 bufFull = false;
123                 Serial.println("time_ms,label,accX,accY,accZ,gyroX,gyroY,gyroZ,detected");
124             } else {
125                 Serial.println("--Recording stopped--");
126             }
127         } else if (reading == HIGH) {
128             buttonHandled = false;
129         }
130     }
131
132     lastButtonState = reading;
133
134     // --- Record data if active ---
135     if (recording) {
136         sensors_event_t a, g, temp;
137         mpu.getEvent(&a, &g, &temp);
138
139         unsigned long t = millis() - startTime;
140
141         // --- Update sliding window ---
142         accY_buf[bufIndex] = a.acceleration.y;
143         accZ_buf[bufIndex] = a.acceleration.z;
144         gyroY_buf[bufIndex] = g.gyro.y;
145
146         bufIndex++;
147

```

```

148     if (bufIndex >= WINDOW_SIZE) {
149         bufIndex = 0;
150         bufFull = true;
151     }
152
153     // --- Classify using the window (once full) ---
154     String detected = "...";
155
156     if (bufFull) {
157         float accY_mean = computeMean(accY_buf, WINDOW_SIZE);
158         float accY_std  = computeStd(accY_buf, WINDOW_SIZE, accY_mean);
159
160         float accZ_mean = computeMean(accZ_buf, WINDOW_SIZE);
161         float accZ_std  = computeStd(accZ_buf, WINDOW_SIZE, accZ_mean);
162
163         float gyroY_ptp = computePeakToPeak(gyroY_buf, WINDOW_SIZE);
164
165         detected = classifyActivity(accY_std, accZ_std, gyroY_ptp);
166     }
167
168     Serial.print(t);
169     Serial.print(",");
170     Serial.print(activityLabel);
171     Serial.print(",");
172     Serial.print(a.acceleration.x, 4);
173     Serial.print(",");
174     Serial.print(a.acceleration.y, 4);
175     Serial.print(",");
176     Serial.print(a.acceleration.z, 4);
177     Serial.print(",");
178     Serial.print(g.gyro.x, 4);
179     Serial.print(",");
180     Serial.print(g.gyro.y, 4);
181     Serial.print(",");
182     Serial.print(g.gyro.z, 4);
183     Serial.print(",");
184     Serial.println(detected);
185
186     delay(50); // ~20 Hz sampling rate
187 }
188 }

```

A.2 Python – Serial Logger with Accuracy Evaluation

```

1 """
2 Serial_logger+accuracy_calculator_for_ESP32+MPU6050_project.
3
4 Works_with_the_Arduino_sketch_that_streams:
5 time_ms,label,accX,accY,accZ,gyroX,gyroY,gyroZ,detected
6
7 Where:
8 -"label" is the ground-truth activity you typed in (via Serial)
9 -"detected" is the real-time rule-based prediction computed on the ESP32
10
11 What this script does:
12 1. Connects to the ESP32 over serial.
13 2. Lets you type a label (e.g. Walking, Running, Up-stairs, Down-stairs) and press

```

```

14 """Enter->it is sent to the ESP32 to set the recorded label.
15 3. Reads all incoming lines, prints them, and saves every valid data row to a CSV
   file.
16 4. On exit (Ctrl+C), computes accuracy by comparing "label" vs "detected" columns,
17 """ and saves a summary CSV with per-activity and overall accuracy.
18
19 How to use:
20 1. Install pyserial and pandas if you don't have them:
21 """ pip install pyserial pandas
22 2. Close the Arduino IDE Serial Monitor (only one program can use the COM port at
   a time).
23 3. Set COM_PORT below to match your Arduino IDE port (e.g. "COM6").
24 4. Run: python serial_logger_with_accuracy.py
25 5. Type a label and press Enter, touch the button wires to start/stop recording,
26 """ do the activity, then type a new label for the next activity, etc.
27 6. Press Ctrl+C when done. Two files will be saved:
28 """- "recorded_data.csv" """-> all raw rows (time, label, acc/gyro values,
   detected)
29 """- "accuracy_summary.csv" """-> accuracy per label + overall accuracy
30 """
31
32 import serial
33 import threading
34 import time
35 import pandas as pd
36
37 COM_PORT = "COM6"          # change this to match your port
38 BAUD_RATE = 115200
39
40 RAW_DATA_FILE = "recorded_data.csv"
41 SUMMARY_FILE = "accuracy_summary.csv"
42
43 stop_flag = False
44 raw_rows = [] # list of parsed dict rows
45
46
47 def reader_thread(ser):
48     while not stop_flag:
49         try:
50             line = ser.readline().decode("utf-8", errors="ignore").strip()
51             if not line:
52                 continue
53
54             print(line) # always show what's coming in
55
56             parts = line.split(",")
57             # Expected data row: time_ms,label,accX,accY,accZ,gyroX,gyroY,gyroZ,
               detected
58             if len(parts) == 9:
59                 try:
60                     row = {
61                         "time_ms": float(parts[0]),
62                         "label": parts[1],
63                         "accX": float(parts[2]),
64                         "accY": float(parts[3]),
65                         "accZ": float(parts[4]),
66                         "gyroX": float(parts[5]),
67                         "gyroY": float(parts[6]),
68                         "gyroZ": float(parts[7]),

```

```

69         "detected": parts[8],
70     }
71     raw_rows.append(row)
72     except ValueError:
73         pass # header row or malformed line, ignore
74 except Exception as e:
75     print(f"Read_error: {e}")
76     break
77
78
79 def save_outputs():
80     if not raw_rows:
81         print("No data collected, nothing to save.")
82         return
83
84     df = pd.DataFrame(raw_rows)
85     df.to_csv(RAW_DATA_FILE, index=False)
86     print(f"\nSaved raw data to {RAW_DATA_FILE}")
87
88     # Keep only rows where a real detection was made (skip the initial "..." warm-
89     # up rows)
90     valid = df[df["detected"] != "..."].copy()
91
92     if valid.empty:
93         print("No valid detection rows found (all were '...'). Accuracy cannot be
94             computed.")
95         return
96
97     valid["correct"] = valid["label"].str.lower() == valid["detected"].str.lower()
98
99     summary_rows = []
100     overall_total = len(valid)
101     overall_correct = valid["correct"].sum()
102
103     for label, group in valid.groupby("label"):
104         total = len(group)
105         correct = group["correct"].sum()
106         acc = 100.0 * correct / total if total > 0 else 0
107         summary_rows.append({
108             "label": label,
109             "total_windows": total,
110             "correct": correct,
111             "accuracy_percent": round(acc, 2)
112         })
113
114     summary_rows.append({
115         "label": "OVERALL",
116         "total_windows": overall_total,
117         "correct": overall_correct,
118         "accuracy_percent": round(100.0 * overall_correct / overall_total, 2) if
119             overall_total > 0 else 0
120     })
121
122     summary_df = pd.DataFrame(summary_rows)
123     summary_df.to_csv(SUMMARY_FILE, index=False)
124
125     print(f"Saved accuracy summary to {SUMMARY_FILE}\n")
126     print(summary_df.to_string(index=False))

```

```

125
126 def main():
127     global stop_flag
128
129     ser = serial.Serial(COM_PORT, BAUD_RATE, timeout=1)
130     time.sleep(2)
131
132     print(f"Connected to {COM_PORT}.")
133     print("Type a label (Walking, Running, Up-stairs, Down-stairs) and press Enter.")
134     print("Touch the button wires together briefly to start/stop recording.")
135     print("Press Ctrl+C to stop and save results.\n")
136
137     t = threading.Thread(target=reader_thread, args=(ser,), daemon=True)
138     t.start()
139
140     try:
141         while True:
142             label = input()
143             if label.strip():
144                 ser.write((label.strip() + "\n").encode("utf-8"))
145                 print(f">> Sent label: {label.strip()}")
146     except KeyboardInterrupt:
147         print("\nStopping...")
148     finally:
149         stop_flag = True
150         time.sleep(0.5)
151         ser.close()
152         save_outputs()
153
154
155 if __name__ == "__main__":
156     main()

```